

Lab 11: Genetic Algorithm – 0/1 Knapsack Problem

Aim

To implement a **Genetic Algorithm (GA)** to solve the **0/1 Knapsack Problem** and find the combination of items that gives the maximum value without exceeding the weight capacity.

Algorithm

1. Define the weights, values, and maximum capacity of the knapsack.
2. Create an initial population of random binary solutions.
3. Calculate the fitness of each solution.
4. Select the best solutions as parents.
5. Apply crossover to generate new solutions.
6. Apply mutation to introduce variation.
7. Repeat the process for a fixed number of generations.
8. Display the best solution, total weight, and total value.

Program

Lab 11: Genetic Algorithm – 0/1 Knapsack Problem

```
import random
```

```
# Item weights and values
```

```
weights = [2, 3, 4, 5]
```

```
values = [3, 4, 5, 6]
```

```
# Maximum knapsack capacity
```

```
capacity = 8
```

```
# Fitness function
```

```

def fitness(solution):
    total_weight = sum(
        solution[i] * weights[i]
        for i in range(len(weights))
    )

    total_value = sum(
        solution[i] * values[i]
        for i in range(len(values))
    )

    # Invalid solution
    if total_weight > capacity:
        return 0

    return total_value

# Create initial population
population = [
    [random.randint(0, 1) for _ in range(len(weights))]
    for _ in range(10)
]

# Run Genetic Algorithm
for generation in range(20):

    # Sort based on fitness
    population = sorted(
        population,
        key=fitness,
        reverse=True
    )

```

```
)
```

```
# Select best parents
```

```
parent1 = population[0]
```

```
parent2 = population[1]
```

```
new_population = [parent1, parent2]
```

```
while len(new_population) < 10:
```

```
    # Crossover
```

```
    point = random.randint(1, len(weights) - 1)
```

```
    child = (
```

```
        parent1[:point] +
```

```
        parent2[point:]
```

```
    )
```

```
    # Mutation
```

```
    if random.random() < 0.2:
```

```
        index = random.randint(0, len(weights) - 1)
```

```
        child[index] = 1 - child[index]
```

```
    new_population.append(child)
```

```
population = new_population
```

```
# Find the best solution
```

```
best_solution = max(population, key=fitness)
```

```
total_weight = sum(
```

```
best_solution[i] * weights[i]
for i in range(len(weights))
)

total_value = fitness(best_solution)

print("Best Solution:", best_solution)
print("Total Weight:", total_weight)
print("Total Value:", total_value)
```

Sample Output

Best Solution: [0, 1, 0, 1]

Total Weight: 8

Total Value: 10

Result

Thus, the **Genetic Algorithm** was successfully implemented to solve the **0/1 Knapsack Problem**, and the optimal combination of items within the given capacity was obtained.